Intelligent Robotics Lab

Universidad Rey Juan Carlos

TU Delft

CORESENSE

Cognitive Architectures
Reliable and Robust Robotics

Esther Aguado

esther.aguado@urjc.es

3rd ACM SIGSOFT Summer School for Software Engineering in Robotics

Table of Contents

1. Introduction

2. Cognitive Architectures

3. Types of Cognitive Architectures
   - Classical
     - SOAR
     - ACT-R
   - Recent
     - CRAM
     - SayCan

4. Metacognition

5. Conclusions

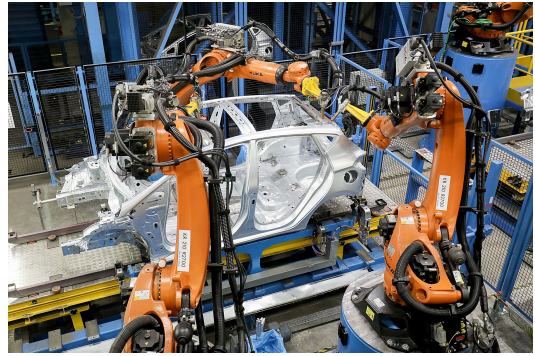
About Me

- Assistant Professor at URJC
- Research in intelligent, robust robotic systems
- PhD in Automatic Control and Robotics from UPM
- Visiting researcher at TU Delft Cognitive Robotics Lab

Why cognitive systems?

Controlled environments

- Environment characteristics are known and predictable
- User needs are clearly defined
- Task conditions are well specified



Controlled environments

- Goal: vacuum and mop the floor efficiently
- Environment mostly known: furniture, walls, décor
- Requirements: safety (robot, people), cleaning quality



Uncontrolled environments

- Goal: tidy up a room
- Environment mostly known but dynamic: furniture, walls, plants, a cat
- Challenges:
  - Which object should be picked up first?
  - What if the furniture has been moved?
  - What if an object does not fit in its intended place?
  - What if an object is inside another object?
    (e.g., a wallet inside a pair of trousers)



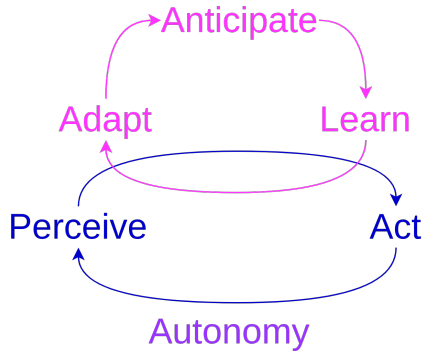
Definitions

Definition of a Cognitive System

Definition

An autonomous system capable of

- **perceiving** its environment,
- **learning** from experience,
- **anticipating** the outcomes of events,
- **acting** to achieve its goals,
- **adapting** to changing conditions.



D. Vernon, *Artificial Cognitive Systems: A Primer*, MIT Press, 2014.

Ideal cognitive robot:

Capable of acting in a changing world,
*knowing **what** it is doing and **why**.*

Definition of a Cognitive Architecture

Definition

A cognitive architecture is a framework that integrates the basic capabilities that allow a robot to understand and operate in the world

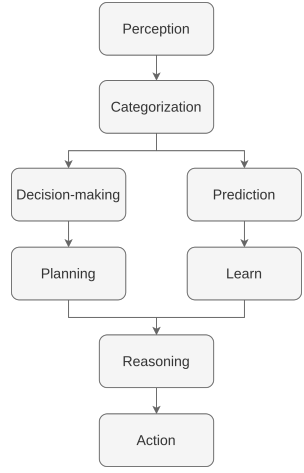
It provides:

- **Memory** (short- and long-term) to store beliefs, goals, and knowledge about the world and itself
- **Representations**: models of the robot, world, and task that allow it to interpret inputs and abstract relevant information from different sources
- **Functional processes**: mechanisms acting on those representations (learning, reasoning, planning, generalisation, . . .)

Core Capabilities of Cognitive Architectures

Cognitive architectures integrate a common set of capabilities that operates on knowledge:

- **Acquisition**: Perception, Attention, categorisation
- **Usage**: Decision-making, planning
- **Analyses**: Adaptation, prediction, monitoring, learning



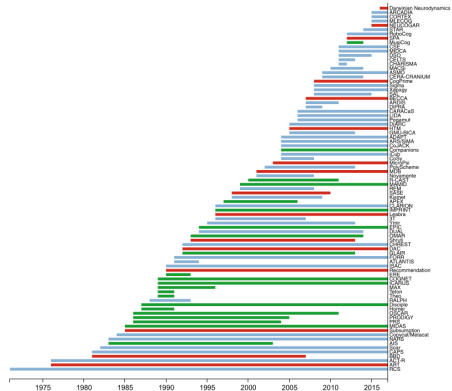
Langley et al., *Cognitive Architectures: Research Issues and Challenges*, 2006.

Perspectives in cognitive architectures

Three types of cognitive architectures

Types:

- **Symbolic**: focus on stable aspects of cognition, model independent of domain and task, logical processing
- **Emergent**: knowledge acquired through experience, reach a fully cognitive state over the system's lifecycle
- **Hybrid**



I. Kotseruba and J. Tsotsos. *40 years of cognitive architectures*. Artificial Intelligence Review, 53(1), 2020.

From Planning to Cognitive Architectures

Classical planners answer:

What sequence of actions achieves my goal?

Cognitive architectures address the question:

How should an autonomous agent act (good enough) in a changing world?

From Planning to Cognitive Architectures

With classical planning:

- The world is *discrete* and *fully known*
- The plan is generated *before* acting
- If something fails $\rightarrow$ replan from scratch
- No memory of past experiences
- No learning

Example:

The robot must fetch the red mug from the table.

- What if the mug is not where expected?
- What if the table is occupied?
- What if the robot has done this task many times and could remember what usually works?

Classical Cognitive Architectures

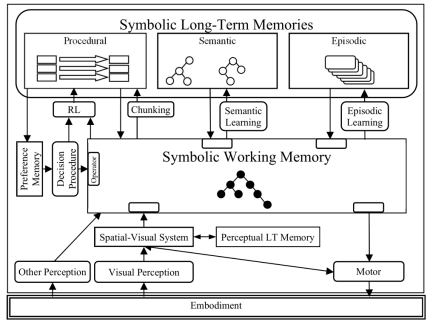
SOAR

State, Operator, And Result

Developed by Newell, Laird and Rosenbloom (1983 – present)

SOAR (State, Operator, And Result)

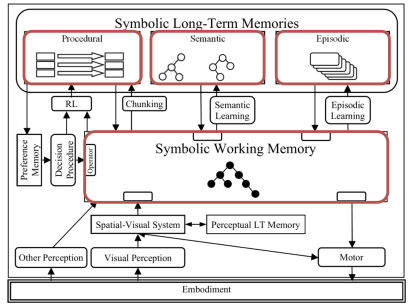
- Aims to model **general intelligent behaviour**, rather than solving a single task
- Instead of executing a fixed plan, the agent **reasons continuously** while interacting with its environment
- Every decision is based on the current state, available knowledge and active goals



Intelligence emerges from a continuous **reasoning cycle**, not from a precomputed plan.

SOAR: Memory Organization

- **Procedural memory**: *how to do things*
  "If gripper is empty and object is accessible, propose pick-up(object)"
- **Semantic memory**: *facts about the world*
  "Mugs with handles should be grasped from the side"
- **Episodic memory**: *past experiences*
  "Last time I used top-grasp on a mug, I dropped it"
- **Working memory**: Current beliefs, goals and candidate actions



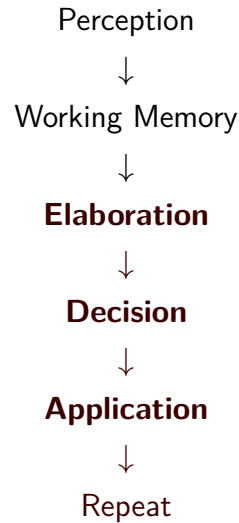
SOAR: The Decision Cycle

The agent continuously repeats three phases:

Elaboration Production rules fire in parallel to update working memory with beliefs, goals and candidate operators

Decision The proposed operators are evaluated and one is selected. If no operator can be selected, SOAR enters an **impasse**

Application The selected operator modifies the current state of the world, and the cycle starts again



SOAR: Impasses

An **impasse** occurs when the agent cannot select the next operator

Typical situations:

- No valid operator is available
- Several operators are equally preferred
- The selected operator cannot be applied

Instead of failing, SOAR creates a **subgoal** to reason about the problem and how to solve it

Example

Goal:
`Fetch(red-mug)`

Possible operators:

- Top grasp
- Side grasp

$\rightarrow$ Tie impasse

SOAR: Learning through Chunking

When an impasse is resolved, SOAR automatically creates a new production rule (*chunk*).

$$\text{Problem} \rightarrow \text{Reasoning} \rightarrow \text{Solution} \rightarrow \text{New Rule}$$

Next time, the learned rule fires **immediately**, avoiding the same deliberation.

SOAR in robotics: Applications

- **ROBO-SOAR (1991)**
  Navigation and manipulation in unstructured environments
- **TacAir-SOAR (1999)**
  Virtual pilot for air combat simulation (USAF)
  Real-time decision-making under uncertainty
- **Quake-Bot (2000)**
  Autonomous agent in video games; learning via chunking in dynamic environments
- **Rosie** (ongoing)
  Domestic service robot controlled by natural language instructions

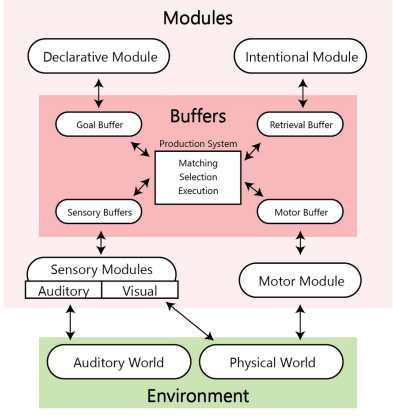
ACT-R

Adaptive Control of Thought – Rational

John Anderson (1993)

ACT-R

- Models **how humans think**, rather than only how intelligent systems behave
- Predicts behaviour but also human features such as reaction times, errors and learning
- Widely used in cognitive psychology and HRI
- SOAR models *general problem solving* while ACT-R models specific human *cognitive tasks*

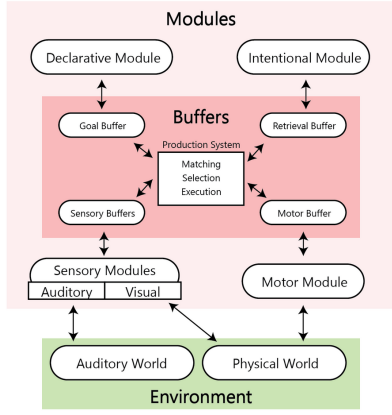


ACT-R: Modules and Buffers

Different cognitive functions are implemented by specialised modules.

- **Visual buffer**: what the agent perceives right now
- **Motor buffer**: what action it is executing
- **Declarative buffer**: what fact it is retrieving from LTM
- **Goal buffer**: what it is trying to achieve
- **Procedural module (central)**: reads all buffers and fires *one rule per cycle*

Each module communicates through a dedicated *buffer*.



ACT-R: Utility learning

When multiple rules can fire, ACT-R selects the one with **highest utility**:

Rule utility

$U = P \cdot G - C$

- P: probability of success if applied
- G: value of the goal
- C: expected cost (time)

Utilities **update with experience**:
if a rule succeeds, its P increases $\rightarrow$ the agent learns which strategies are more efficient

Recent Cognitive Architectures

CRAM

Cognitive Robot Abstract Machine

Beetz et al., University of Bremen (2010 – present)

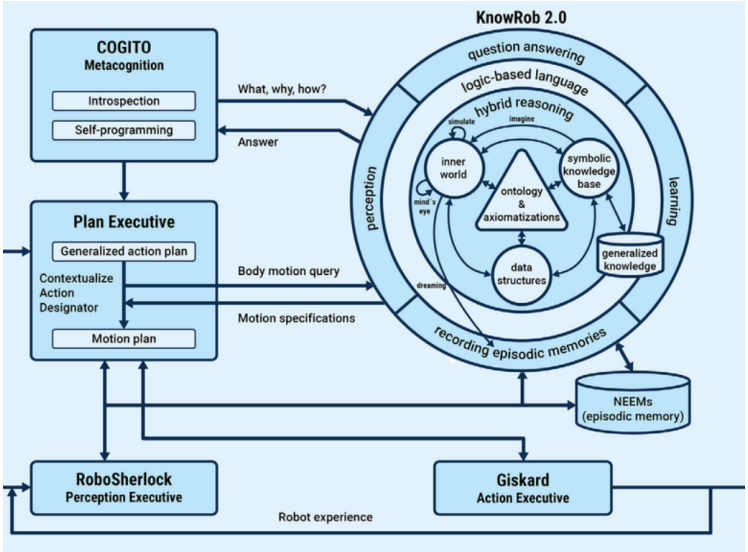
CRAM: Cognitive Robot Abstract Machine

Hybrid architecture for *everyday activities* in domestic environments.

Input: *"make a crepe"* $\rightarrow$ Output: fully parametrised motion plan
Five coordinated modules:

- **CPL**: Lisp-based plan executive; monitors and adapts at runtime
- **KnowRob**: knowledge representation
- **NEEMs**: Narrative Enabled Episodic Memories
- **RoboSherlock**: semantic perception (6-DOF pose, occlusions)
- **Giskard**: motion execution via optimisation + RL
- **COGITO**: metacognition: *"why did we fail?"*

CRAM: Cognitive Robot Abstract Machine



CRAM: Failure Handling and Metacognition

When something unexpected happens (object not found, grasp failed):

- **CPL registers** the failure
- **COGITO explains** what went wrong
  "Did the robot not see the mug?"
  "Was the grasp pose wrong?"
- **KnowRob is queried** for alternatives
- **CPL adapts** the plan and retries



SayCan

Do As I Can, Not As I Say

Google Robotics, 2022

SayCan: Do As I Can, Not As I Say

The problem: LLMs have rich semantic knowledge but are *not grounded* in the physical world.

- An LLM knows that *"I spilled my drink"* implies cleaning up
- But it does not know what *this robot* can do in *this environment*
- It might suggest a vacuum cleaner even if the robot has none

SAY: task grounding (LLM)

Analyses the instruction.
Assigns a **utility score** to each skill.

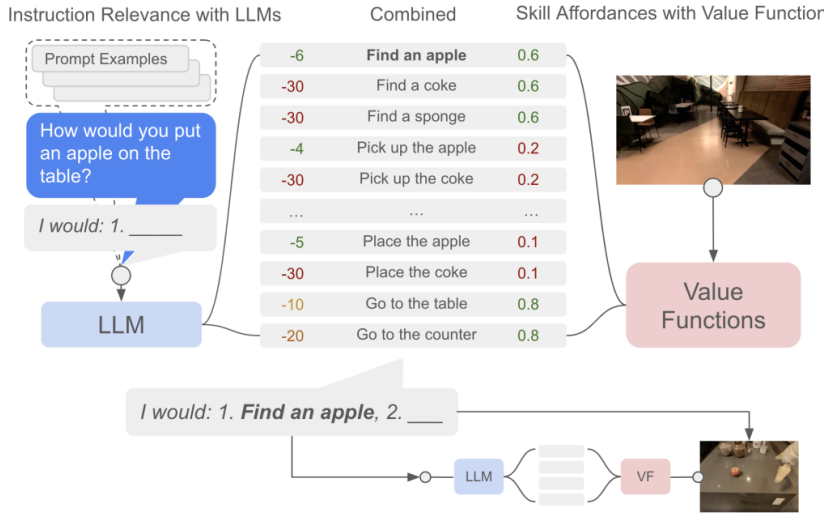
CAN: world grounding (RL)

Value functions encode physical **affordances**.
Assigns a **feasibility score** to each action in the current state.

The solution: ground the LLM with physical *affordances*.

Selected action: $\arg\max\ p_{\text{utility}} \times p_{\text{feasibility}}$

SayCan: Do As I Can, Not As I Say



SayCan: Results and Limitations

101 instructions across 7 task families: mobile manipulator in kitchen environments.

Results

- Planning success: **84%** (lab), **81%** (real kitchen)
- Execution success: **74%** (lab), **60%** (real kitchen)
- Without value functions: only **67%** planning $\rightarrow$ lacks physical grounding
- 65% of errors are LLM failures, 35% affordance failures

Limitations

- *Skill range is the bottleneck*: the robot can only do what it has been trained to do
- Natural-language flexibility that classical architectures lack
- But: no episodic memory, no explanation of failure, no awareness of its own reasoning

SayCan knows *what* to do next, but not *why* it failed, nor whether it is reasoning well.

Metacognition

Metacognition

Metacognition

Reflect on internal processes (resources, confidence, progress).

Enables *self-awareness* and *adaptive decision-making*.

In practice, a metacognitive robot asks:

- *Is my perception reliable right now?*
- *Am I uncertain about the next action?*
- *Is my plan still valid?*
- *Did something go wrong, why?*

Cognition

Perceive → Reason → Act

↓

Metacognition

Monitor → Evaluate → Reflect

↓

Metacontrol

Analize → Decide/Plan → Adapt

Metacontrol

Metacognition observes:

- I am uncertain
- My perception is unreliable
- I do not know which object to grasp
- My plan may fail
- I recognise a previously encountered situation

Metacontrol decides:

- Gather more information
- Switch perception pipeline
- Switch planning strategy
- Ask the user
- Replan or abandon the plan
- Learn from this failure

Metacontrol in the Architectures

System	How it handles not knowing what to do next
SOAR	An **impasse** triggers a subgoal: deliberates by creating a subgoal until operator selection succeeds. Resolved knowledge is stored as a *chunk*: the same problem never requires the same deliberation twice.
ACT-R	**Utility learning** shifts rule preferences based on past success and cost: adapts behaviour without explicit metacognitive reasoning.
CRAM	**COGITO** monitors execution, diagnoses failures, and supports adaptive replanning via KnowRob.
SayCan	Scores the next action but cannot reason about why its own decision process failed.

CORESENSE

A hybrid cognitive architecture

For deep understanding and awareness

The CoreSense project has received funding from the European Union's Horizon Europe research and innovation programme under grant agreement No. 101070054

coresense.eu

Conclusions

- Cognitive architectures offer **complementary views** of intelligence
- Each system solves part of the problem from a perspective:
  SOAR's memory, ACT-R's human modelling, CRAM's knowledge representation, SayCan's language grounding
- The hardest part is **integrating** modules that support generality

*A cognitive robot should know **what** it is doing and **why**.*

Open Challenges

- **Grounding**: Connecting symbols with sensory experience
- **Long-term memory**: Continuous learning, temporal reasoning, episodic management
- **Scalability**: Real-time reasoning with large knowledge bases
- **Evaluation**: No standardised benchmarks for cognitive behaviour

Intelligent Robotics Lab

Universidad Rey Juan Carlos

TU Delft

CORESENSE

Cognitive Architectures
Reliable and Robust Robotics

Esther Aguado

esther.aguado@urjc.es

3rd ACM SIGSOFT Summer School for Software Engineering in Robotics

Tutorial

Repository

`https://github.com/estherag/intro_cognitive_architectures`

- **Exercise 1: Symbolic Reasoning** Explicit knowledge representation and logical inference
- **Exercise 2: Knowledge Management** Beliefs, uncertainty and memory
- **Exercise 3: Goal Management** Goal monitoring, decision making and semantic memory

Perception → Knowledge → Reasoning → Memory → Goals → Action